

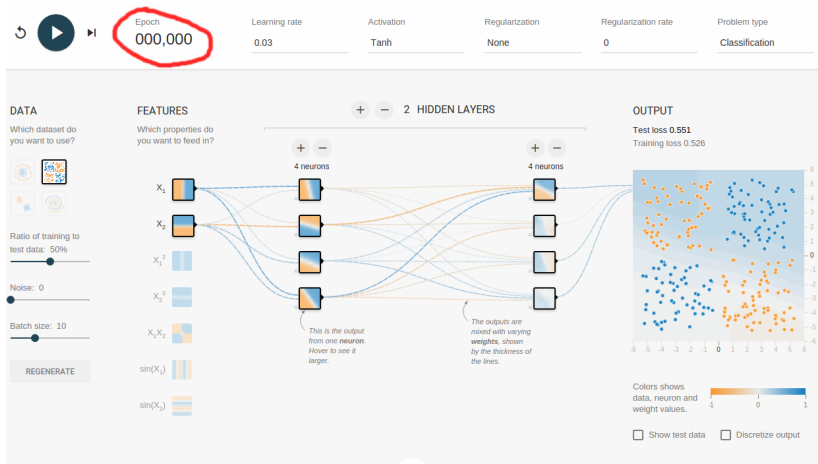
Jaki mają związek sieci neuronowe z łamigłówką Burze i inne pytania

Paweł Rychlikowski

Instytut Informatyki UWr

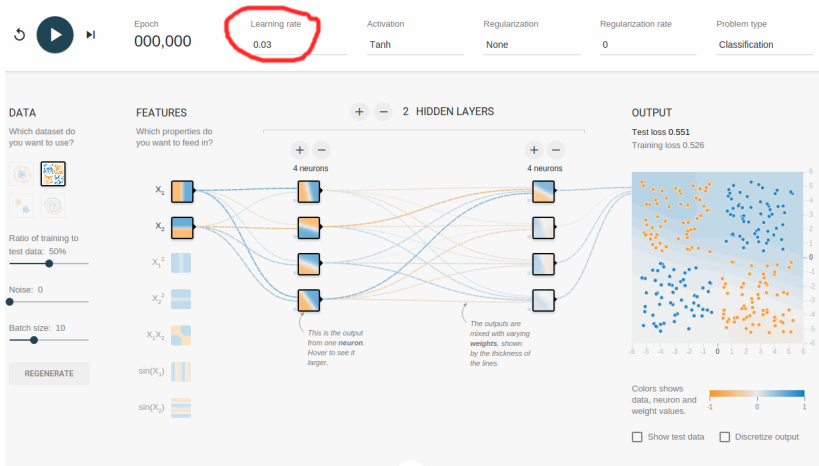
28 maja 2026

Plac zabaw dla tensorflow. Ważne pojęcia (1)



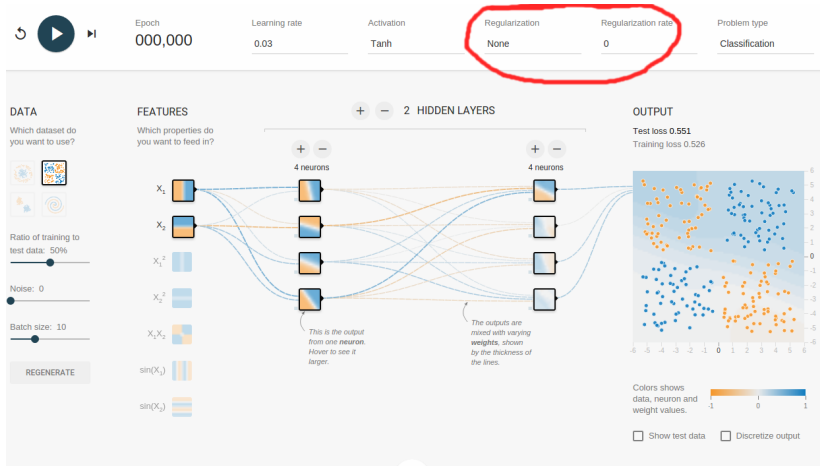
Epoka: etap uczenia, w którym uwzględnione są wszystkie dane uczące.

Plac zabaw dla tensorflow. Ważne pojęcia (2)



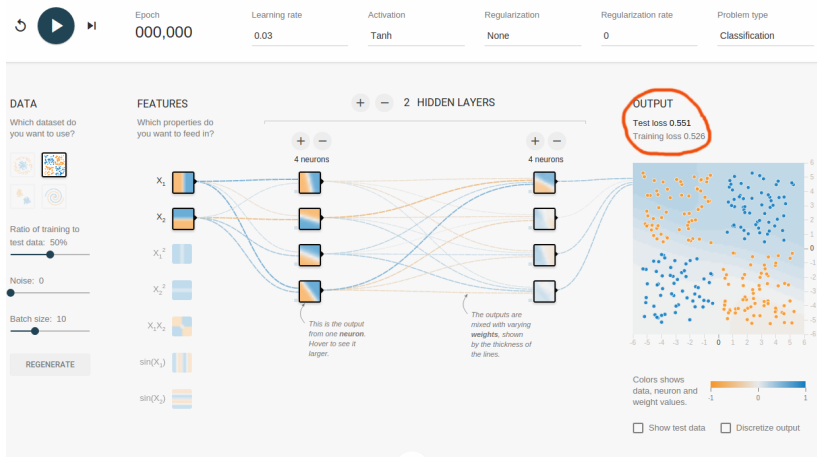
Learning rate: stała przez którą mnożone są **delty** wag. Za duża może dać chaotyczne zachowanie, za mała: bardzo wolny postęp.

Plac zabaw dla tensorflow. Ważne pojęcia (3)



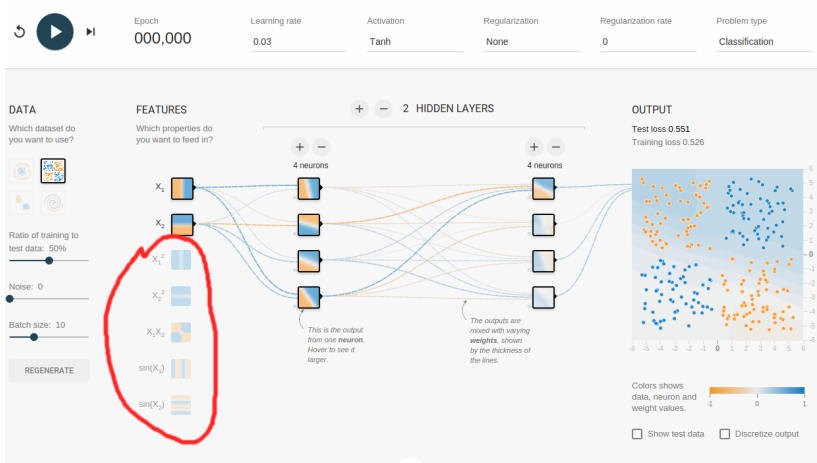
Regularyzacja: dołożenie do uczenia wymagania, by wagi nie były zbyt duże. Może dać większą stabilność uczenia (zob. tablica).

Plac zabaw dla tensorflow. Ważne pojęcia (4)



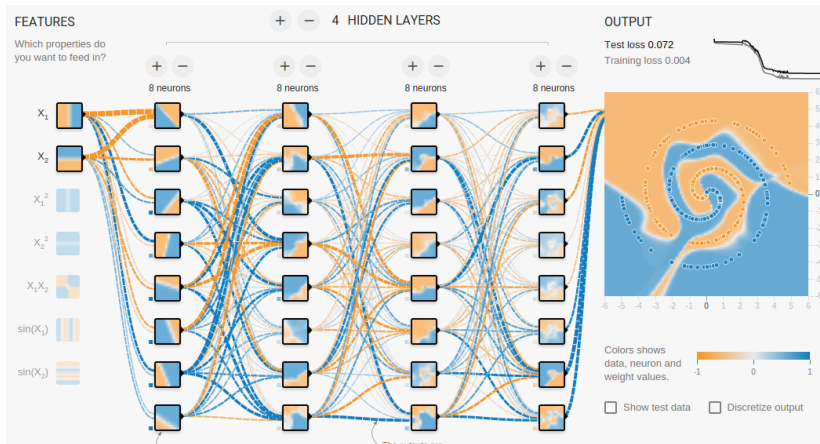
Test loss/training loss: wartość funkcji straty dla zbioru testowego i uczącego (oczywiście pierwsza zawsze większa).

Plac zabaw dla tensorflow. Ważne pojęcia (5)



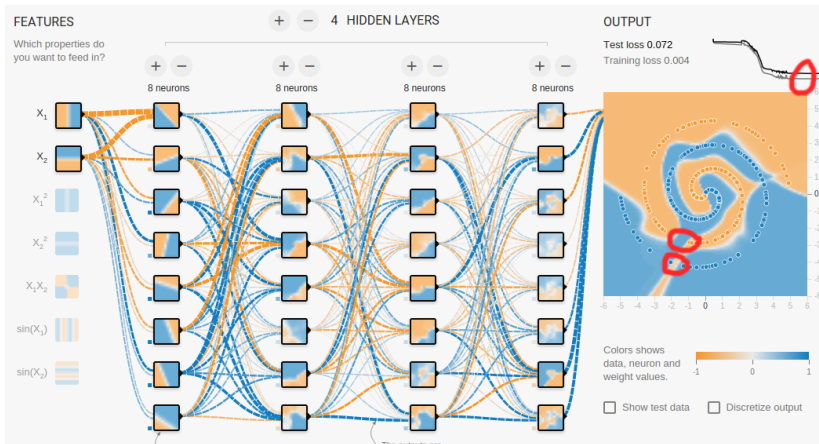
Feature engineering: proces tworzenia własnych cech dla konkretnych przypadków. Dobre cechy mają **związek z zadaniem**.

Plac zabaw dla tensorflow. Ważne pojęcia (6)



Przeuczenie (overfitting): sytuacja, w której sieć dostosowuje się do **nieistotnych** fluktuacji danych uczących, co pogarsza generalizację.

Plac zabaw dla tensorflow. Ważne pojęcia (6)



Przeuczenie (overfitting): sytuacja, w której sieć dostosowuje się do **nieistotnych** fluktuacji danych uczących, co pogarsza generalizację.

- Wejściem do sieci jest **wektor** (czyli ciąg liczb o ustalonej długości)

- Wejściem do sieci jest **wektor** (czyli ciąg liczb o ustalonej długości)
- W tym wektorze możemy zakodować wszystko:
 - obrazki (jak?)
 - teksty o ustalonej długości (jak?)
 - sytuację na planszy w Reversi (jak?)

Kodowanie wejścia

- Wejściem do sieci jest **wektor** (czyli ciąg liczb o ustalonej długości)
- W tym wektorze możemy zakodować wszystko:
 - obrazki (jak?)
 - teksty o ustalonej długości (jak?)
 - sytuację na planszy w Reversi (jak?)

Kodowanie **one-hot**

Sieci neuronowe lubią *rozwlekle* kodowanie, w którym liczbę $i \in \{0, \dots, N - 1\}$ kodujemy jako $(0, 0, 0, \dots, 1, \dots, 0, 0)$ (jedyńska na i -tej pozycji).

- Pamiętajmy, że możemy dowolnie tworzyć cechy dla przypadków testowych:
 - Kwantyzacja dla obrazów
 - Analiza Fouriera dla dźwięków
 - ...

- Pamiętajmy, że możemy dowolnie tworzyć cechy dla przypadków testowych:
 - Kwantyzacja dla obrazów
 - Analiza Fouriera dla dźwięków
 - ...

Uwaga

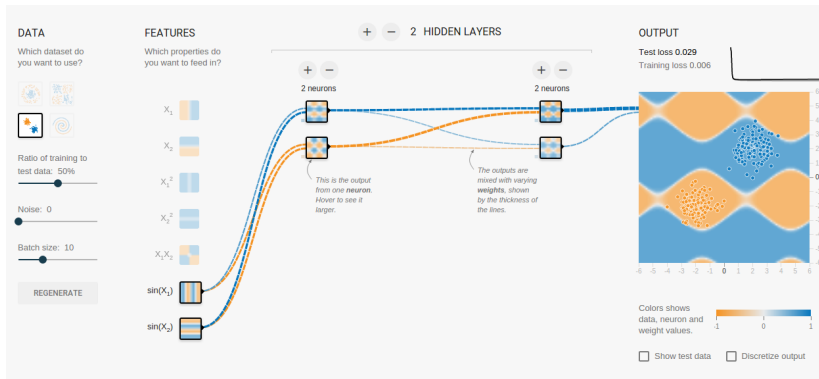
Dodając cechy możemy przyspieszyć uczenie, ale możemy też *zasugerować* sieci naszą wizję świata.

- Pamiętajmy, że możemy dowolnie tworzyć cechy dla przypadków testowych:
 - Kwantyzacja dla obrazów
 - Analiza Fouriera dla dźwięków
 - ...

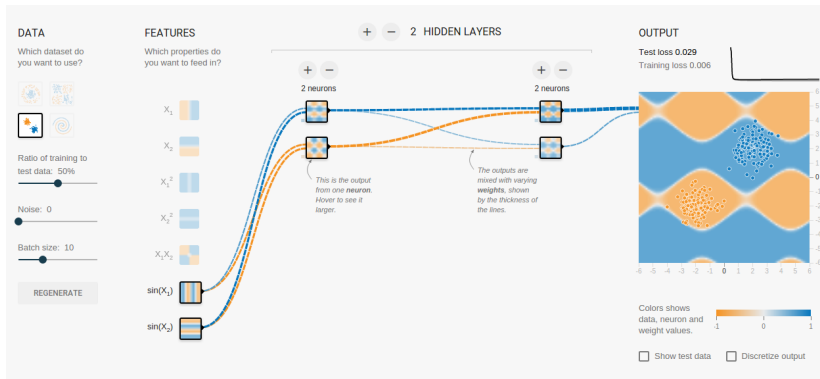
Uwaga

Dodając cechy możemy przyspieszyć uczenie, ale możemy też *zasugerować* sieci naszą wizję świata. Np. cecha w Reversi: *wynik jakiejś funkcji heurystycznej*.

Sugerowanie cykliczności



Sugerowanie cykliczności



Sieć w miarę poprawnie sklasyfikowała zbiór uczący, dobrze też go uogólnia, ale jest przekonana, że świat jest mozaiką. Nikt z nas, widząc te dane nie wyrobił sobie tego poglądu.

- Często chcemy, żeby sieć decydowała o jednej z K opcji (zadanie klasyfikacji).

- Często chcemy, żeby sieć decydowała o jednej z K opcji (zadanie klasyfikacji).
- **Rozmywamy** ten wybór, prosząc o podanie rozkładu prawdopodobieństwa dla wszystkich K opcji.

- Często chcemy, żeby sieć decydowała o jednej z K opcji (zadanie klasyfikacji).
- Rozmywamy ten wybór, prosząc o podanie rozkładu prawdopodobieństwa dla wszystkich K opcji.
- To tzw. **Softmax layer**, która przypisuje prawdopodobieństwo zależne od wielkości pobudzenia.

- Często chcemy, żeby sieć decydowała o jednej z K opcji (zadanie klasyfikacji).
- Rozmywamy ten wybór, prosząc o podanie rozkładu prawdopodobieństwa dla wszystkich K opcji.
- To tzw. **Softmax layer**, która przypisuje prawdopodobieństwo zależne od wielkości pobudzenia.

Wzór:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^d e^{z_j}}$$

- Często chcemy, żeby sieć decydowała o jednej z K opcji (zadanie klasyfikacji).
- **Rozmywamy** ten wybór, prosząc o podanie rozkładu prawdopodobieństwa dla wszystkich K opcji.
- To tzw. **Softmax layer**, która przypisuje prawdopodobieństwo zależne od wielkości pobudzenia.

Wzór:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^d e^{z_j}}$$

Popatrzmy na to, jak działa funkcja Softmax.

```
>>> softmax([1,2,3])  
[0.09003, 0.24472, 0.66524]
```

```
>>> softmax([1,2,3,10])  
[0.00012, 0.00033, 0.00091, 0.99863]
```

```
>>> softmax([3,4,4])  
[0.15536, 0.42231, 0.42231]
```

Super łatwe sieci neuronowe

- Można wykorzystać bibliotekę **sklearn** (lub analogiczną), która implementuje **MLP** (czyli wielowarstwowy perceptron)

Super łatwe sieci neuronowe

- Można wykorzystać bibliotekę **sklearn** (lub analogiczną), która implementuje **MLP** (czyli wielowarstwowy perceptron)
- Sieć definiujemy jednym konstruktorem z dużą liczbą parametrów (ale ufamy, że wartości domyślne są ok)

Super łatwe sieci neuronowe

Przygotowanie danych

```
from sklearn.neural_network import MLPClassifier
import random, pickle
```

```
# data: list of pairs (X,y)
# X[i]: vector of floats/ints
# y[i] in [v1,...,vk]
```

```
random.shuffle(data)
N = len(data) / 6
test_data = data[:N]
dev_data  = data[N:]
```

```
X = [x for (x,y) in dev_data]
y = [y for (x,y) in dev_data]
X_test = [x for (x,y) in test_data]
y_test = [y for (x,y) in test_data]
```

Super łatwe sieci neuronowe (2)

Uczenie sieci

```
# creating model
nn = MLPClassifier(hidden_layer_sizes=(60,60,10))

# training model
nn.fit(X,y)

print ('Dev_score', nn.score(X,y))
print ('Test_score', nn.score(X_test, y_test))

# writing model
with open('nn_weights.dat', 'w') as f:
    pickle.dump(nn, f)
```

Super łatwe sieci neuronowe (3)

Korzystanie z sieci

```
from sklearn.neural_network import MLPClassifier
import pickle

with open('nn_weights.dat') as f:
    nn = pickle.load(open(f))

x = data_vector

probabilities = nn.predict_proba([x])

prob0 = probabilities[0][0]
prob1 = probabilities[0][1] # for two classes 0 and 1
```

Cons

- Oczywiście daje dużo mniejszą swobodę niż bardziej specjalizowane biblioteki.

Cons

- Oczywiście daje dużo mniejszą swobodę niż bardziej specjalizowane biblioteki.
- Nadaje się do tworzenia niezbyt dużych sieci

Cons

- Oczywiście daje dużo mniejszą swobodę niż bardziej specjalizowane biblioteki.
- Nadaje się do tworzenia niezbyt dużych sieci
- Nie ma sieci spłotowych, sieci rekurencyjnych, ...

Cons

- Oczywiście daje dużo mniejszą swobodę niż bardziej specjalizowane biblioteki.
- Nadaje się do tworzenia niezbyt dużych sieci
- Nie ma sieci splotowych, sieci rekurencyjnych, ...

Pros

- Bardzo prosta w użyciu i wystarczająco szybka

Cons

- Oczywiście daje dużo mniejszą swobodę niż bardziej specjalizowane biblioteki.
- Nadaje się do tworzenia niezbyt dużych sieci
- Nie ma sieci spłotowych, sieci rekurencyjnych, ...

Pros

- Bardzo prosta w użyciu i wystarczająco szybka
- Ten sam (prawie) interfejs dla różnych mechanizmów:
 - `from sklearn.neighbors import KNeighborsClassifier as Classifier`
 - `from sklearn.tree import DecisionTreeClassifier as Classifier`
 - `from sklearn.svm import SVC as Classifier`
 - ... (i jeszcze kilkanaście innych)

Funkcja kosztu

Definicja

Funkcja **kosztu** (loss) opisuje, jak bardzo **niezadowoleni** jesteśmy z działania naszego mechanizmu (klasyfikatora, przewidywacza wartości).

Definicja

Funkcja **kosztu** (loss) opisuje, jak bardzo **niezadowoleni** jesteśmy z działania naszego mechanizmu (klasyfikatora, przewidywacza wartości).

Funkcja kosztu jest określona na: danych uczących (x, y) oraz wagach (parametrach klasyfikatora). Przy czym dane uczące traktujemy jako parametr, a wagi – jako właściwe argumenty.

Definicja

Funkcja **kosztu** (loss) opisuje, jak bardzo **niezadowoleni** jesteśmy z działania naszego mechanizmu (klasyfikatora, przewidywacza wartości).

Funkcja kosztu jest określona na: danych uczących (x,y) oraz wagach (parametrach klasyfikatora). Przy czym dane uczące traktujemy jako parametr, a wagi – jako właściwe argumenty.

Przykładowa funkcja kosztu: **błąd średniokwadratowy**

Funkcja kosztu (2)

Średniokwadratowa funkcja straty

$$\text{TrainLoss}(w) = \frac{1}{|D_{\text{train}}|} \sum_{(x,y) \in D_{\text{train}}} (f_w(x) - y)^2$$

Średniokwadratowa funkcja straty

$$\text{TrainLoss}(w) = \frac{1}{|D_{\text{train}}|} \sum_{(x,y) \in D_{\text{train}}} (f_w(x) - y)^2$$

Wariant liniowy:

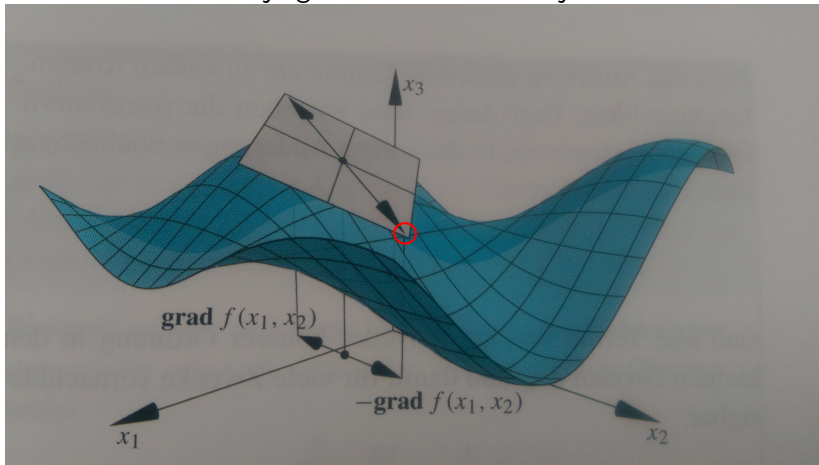
$$f_w(x) = \sum_{i=0}^N w_i \phi_i(x)$$

- Gradient jest wektorem pochodnych cząstkowych.
- Wskazuje kierunek największego wzrostu funkcji.

- Gradient jest wektorem pochodnych cząstkowych.
- Wskazuje kierunek największego wzrostu funkcji.

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

Wizualizacja gradientu w dla funkcji $\mathcal{R}^2 \rightarrow \mathcal{R}$



Zadanie

Wyrazić problem szukania rozwiązania zagadki Burze jako problem znalezienia minimum pewnej różniczkowalnej funkcji

Zadanie

Wyrazić problem szukania rozwiązania zagadki Burze jako problem znalezienia minimum pewnej różniczkowalnej funkcji

- Użyjemy biblioteki pytorch, która da nam automatyczne różniczkowanie oraz zaimplementowany wzorek na mądrzejszą optymalizację (2 następne slajdy)
- Popatrzymy na kod `storms1.py` i `storms2.py`